TP n°0 – Création et préparation des jeux de données

HAMMAL Zakaria TRIBECHE Rayan NAIT MIHOUB Imad
FERGUENE Abderraouf

4 novembre 2026

Table des matières

Introduction **2**

1 Random 100 **2**

2 Random 1000 **2**

3 Test-1 **2**

4 Test-2 **3**

5 Test-3 **3**

6 Caractéristiques des jeux de données **3**

Conclusion **4**

Introduction

L'objectif de ce premier TP est de préparer les jeux de données qui serviront, dans le TP n°1, à comparer quatre algorithmes de test de primalité. Pour que la comparaison soit pertinente, il faut des jeux de données variés : petits nombres, grands nombres, distributions contrôlées par taille, etc.

Un point essentiel de notre démarche est que l'on cherche à obtenir une **véritable randomness** dans les jeux de données, et donc on n'a **pas de reproductibilité** : chaque exécution du générateur doit produire un jeu différent, ce qui assure que les algorithmes du TP n°1 seront testés sur des données réellement variées et non sur un échantillon préparé à l'avance. Une fois les cinq fichiers générés, on les garde tels quels sur le disque : ce sont ces fichiers-là qui seront utilisés par A1, A2, A3 et A4 dans le TP n°1, pour que la comparaison des temps d'exécution soit valide (les quatre algorithmes tournent ainsi sur les mêmes données).

Pour la génération, on utilise dix fichiers de nombres premiers provenant du site http://www.prime-numbers.org/, notés `primes10.txt` et `lino1` à `lino9`. Ces fichiers contiennent chacun une liste de nombres premiers, un par ligne, triés dans l'ordre croissant.

Cinq jeux de données sont préparés :

— `Random100` et `Random1000` : nombres premiers tirés au hasard dans un vaste corpus ;
— `Test-1` : 100 nombres premiers de l'intervalle $[1000, 10^6]$;
— `Test-2` : 10 nombres premiers de chaque taille (3, 4 et 5 chiffres) ;
— `Test-3` : 10 nombres premiers de chaque taille (3, 6, 9 et 12 chiffres).

1 Random 100

Pour générer un jeu de données qui contient 100 nombres premiers, on utilise une variable aléatoire pour choisir le fichier à partir duquel on tire le nombre, puis on utilise une autre variable qui simule ń Pile ou Face ż pour décider si on prend le numéro ou non. Si on décide bien de prendre un nombre, on décrémente un compteur initialisé à 100, **cpt**, pour signaler qu'on a bien choisi un nombre : il nous reste alors $\texttt{cpt} \leftarrow \texttt{cpt} - 1$.

On répète cette opération jusqu'à ce que $\texttt{cpt} = 0$.

Dans cette approche, on ne connaît pas à l'avance le nombre de répétitions, car à chaque répétition on choisit d'une manière aléatoire si on prend le nombre ou non ; mais vu que la taille des fichiers est très grande et dépasse 10^{10} nombres, on peut être quasi sûr que l'on va choisir 100 nombres avant la fin d'un fichier.

Comme on garde la randomness, on ne cherche pas à reproduire exactement le même `Random100` d'une exécution à l'autre : c'est un choix assumé.

2 Random 1000

On utilise la même approche que pour **Random100**, mais on initialise le compteur à 1000.

3 Test-1

Dans ce dataset, les nombres sont compris entre 1000 et 1000000, donc on utilise le fichier **primes10.txt**.

1. D'abord, on lit les 168 premiers nombres du fichier pour les dépasser, car on sait qu'il y a 168 nombres premiers inférieurs à 1000, et ces nombres ne nous intéressent pas.
2. On sait qu'il y a 78498 nombres premiers inférieurs à 10^6, donc on a $78498 - 168 = 78330$ nombres à partir desquels on choisit 100 nombres. Cette taille est petite relativement au nombre initial de nombres premiers dans **primes10.txt**.

3. On utilise deux tableaux : un pour sauvegarder les nombres qui nous intéressent, et l'autre pour vérifier si un nombre a déjà été choisi.
4. À chaque fois, on choisit un nombre au hasard dans le tableau ; s'il n'a pas déjà été choisi, on l'ajoute à notre fichier de sortie et on le marque comme choisi, puis on décrémente le compteur ; sinon on répète.
5. On s'arrête quand le compteur atteint 0.

Dans cette approche, on ne connaît pas à l'avance le nombre de répétitions, car à chaque répétition on choisit d'une manière aléatoire ; mais vu que la taille de l'ensemble de choix est très petite par rapport aux 78330 candidats, on peut être quasi sûr que l'on va choisir 100 nombres distincts dans un temps raisonnable.

4 Test-2

Dans ce dataset, on utilise une approche similaire à l'approche précédente, mais cette fois pour les nombres à 3 chiffres, 4 chiffres et 5 chiffres, et on initialise les compteurs à chaque fois à 10 pour choisir 10 nombres de chaque catégorie.

Les trois intervalles considérés sont :
— 3 chiffres : [100, 999] ;
— 4 chiffres : [1000, 9999] ;
— 5 chiffres : [10000, 99999].

5 Test-3

Si on utilisait un fichier qui contient tous les nombres premiers à 12 chiffres, cela consommerait environ 400 Go du disque (il y a environ $3{,}3 \cdot 10^{10}$ nombres premiers entre 10^{11} et $10^{12} - 1$), alors on utilise un script Python **generate.py** pour générer des nombres premiers du type **1xxxxxxxxxxx** (12 chiffres commençant par 1).

Le script parcourt l'intervalle $[10^{11}, 2 \cdot 10^{11} - 1]$ en sélectionnant des candidats au hasard, puis teste leur primalité par un test de Miller-Rabin déterministe. Les nombres sont ensuite écrits un par ligne dans un fichier texte, au format standard (fin de ligne Unix, pas de BOM, pas de ligne vide), afin d'être lus sans erreur par le programme C.

On répète ensuite les mêmes étapes que pour **Test-1** : lecture séquentielle du fichier, mémorisation des nombres dans des tableaux par catégorie de taille, puis tirage aléatoire de 10 valeurs par catégorie. Les quatre catégories sont :
— 3 chiffres : [100, 999] ;
— 6 chiffres : $[10^5, 10^6 - 1]$;
— 9 chiffres : $[10^8, 10^9 - 1]$;
— 12 chiffres : fichier Python dédié.

6 Caractéristiques des jeux de données

Le tableau suivant récapitule les caractéristiques des cinq jeux de données produits.

Jeu de données	Valeurs	Minimum	Maximum	Distribution
`Random100`	100	2	$\approx 9 \cdot 10^{10}$	hétérogène
`Random1000`	1000	2	$\approx 9 \cdot 10^{10}$	hétérogène
`Test-1`	100	1009	984563	uniforme $[10^3, 10^6]$
`Test-2`	30	131	91733	3, 4, 5 chiffres
`Test-3`	40	149	$\approx 10^{11}$	3, 6, 9, 12 chiffres

TABLE 1 – Récapitulatif des cinq jeux de données produits dans ce TP.

Vérifications effectuées et non effectuées

On n'a pas fait certaines vérifications explicites (cardinalité exacte de chaque fichier, primalité de chaque valeur par un test indépendant, format strict de chaque ligne), car on lance le code de génération seulement une fois et pour des données que l'on connaît. Les dix fichiers sources proviennent de http://www.prime-numbers.org/ et contiennent déjà des nombres premiers au format standard (un nombre par ligne, sans BOM, avec des fins de ligne Unix) ; notre génération ne fait que piocher dedans avec `rand()`, sans modifier les valeurs lues. Le fait de n'exécuter le générateur qu'une seule fois, puis de garder les fichiers tels quels sur le disque, garantit aussi que ce sont exactement les mêmes données qui seront utilisées par A1, A2, A3 et A4 dans le TP n°1.

Les seules vérifications qu'on a faites sont celles qui sont directement visibles à l'œil nu : le nombre de lignes de chaque fichier correspond au nombre attendu (100, 1000, 100, 30 et 40), et les valeurs restent dans les plages attendues (par exemple $[1000, 10^6]$ pour `Test-1`). Pour une utilisation plus rigoureuse, il faudrait refaire une passe de vérification automatique (cardinalité, primalité de chaque valeur par un test de Miller-Rabin indépendant, format strict de chaque ligne), mais ce n'était pas nécessaire ici puisque le code de génération n'a été lancé qu'une seule fois et sur des données dont on connaissait déjà la provenance et la nature.

Conclusion

Ce TP nous a permis de construire cinq jeux de données adaptés aux besoins du TP n°1. Chaque jeu cible un aspect particulier de la comparaison entre algorithmes : `Random100` et `Random1000` illustrent le comportement moyen des algorithmes sur des entrées très variées, ce qui est représentatif d'un usage réel ; `Test-1` illustre le comportement sur une plage de taille modérée et uniforme ; `Test-2` et `Test-3` isolent l'effet de la taille des nombres, grâce à des distributions contrôlées par nombre de chiffres.

La génération elle-même nous a confrontés à plusieurs questions pratiques importantes :

1. **Sources des données.** Il est impossible de générer exhaustivement les nombres premiers à 12 chiffres (plus de $3 \cdot 10^{10}$ valeurs). Il faut donc soit recourir à des fichiers précalculés, soit utiliser un test de primalité probabiliste pour générer les nombres à la volée.
2. **Randomness et reproductibilité.** L'utilisation de `srand(time(NULL))` est un choix délibéré : on cherche à obtenir une véritable randomness dans les jeux de données, ce qui exclut par construction la reproductibilité. Chaque exécution produit un jeu différent, ce qui garantit que les algorithmes du TP n°1 ne sont pas testés sur un échantillon préparé à l'avance, mais sur des données véritablement variées. Une fois les fichiers générés, ils sont gardés tels quels sur le disque : la randomness est introduite au moment de la génération, pas au moment de l'exécution des algorithmes.
3. **Vérification.** Un jeu de données n'est utilisable que s'il est vérifié : cardinalité, primalité de chaque valeur, format strict. C'est un point souvent négligé dans la précipitation.

Les cinq fichiers générés (`Random100`, `Random1000`, `Test-1`, `Test-2`, `Test-3`) sont maintenant prêts et serviront de base expérimentale à l'étude comparative du TP n°1. Ce sont ces fichiers, et

eux seuls, qui seront lus par les quatre algorithmes A1, A2, A3 et A4 : la randomness introduite à la génération garantit que les jeux sont variés, et leur archivage sur disque garantit que les quatre algorithmes seront comparés sur les mêmes données.